

End-to-End MILP Solver Pipeline

Consolidated Execution Order — Integer-Aware Presolve, Root Warm-Start, Cutting Planes, Branch-and-Bound, and Verified Integer-Feasible Solve

Revision 3 — corrects a scaled/unscaled post-solve mixing bug, removes an invalid MILP dual certificate, closes two broken cross-references, adds concrete operational defaults, and hardens the incumbent-snapshot mechanism for thread-safety

Scope. This document specifies the Mixed-Integer Linear Programming (MILP) extension of the LP pipeline, for the same problem statement (PS 26119, Indigenous GPU-Accelerated Optimization Solver, MRPL). It does not re-derive the LP document's internals — it reuses them. Steps 1–6 map directly onto LP Steps 1–6, with only the deltas needed for integrality spelled out; Steps 7–12 are new. This revision closes a review pass that surfaced two genuine correctness defects inherited from the LP document's post-solve stage, two broken internal cross-references, and several under-specified operational parameters.

Companion-document dependency

What is shared, verbatim. Steps 1–6 (ingestion/scaling, presolve, GPU warm-start, crossover, Dual Revised Simplex, degeneracy control) are the LP pipeline's own machinery, called unchanged. Branch-and-Bound is a thin combinatorial shell wrapped around that core, not a re-derivation of it: the GPU warm-start and Crossover are expensive, once-only operations at the root; the Dual Revised Simplex core is cheap to re-enter from a nearby basis at every one of the potentially millions of nodes that follow.

Consistency obligation. Because Steps 1–6 are shared by reference, any correction made to the LP document's Steps 1–6 (including the post-solve fix below, which originates in LP Step 7) must be mirrored there, not just here — the two documents are not independently correct.

Step 1 — Ingestion, Condition Screening & Scaling (CPU) — shared with LP, + integrality tagging

Identical to LP Step 1: MPS/LP parsing into CSC(A)/CSR(A^T), condition screening at the 10⁸ threshold, Ruiz Equilibration, Pock–Chambolle preconditioning. One addition at parse time:

1. **Integrality tagging.** Record the integer index set $I \subseteq \{1, \dots, n\}$ (and the binary subset $B \subseteq I$, since binaries admit stronger presolve and cut machinery than general integers) as a bitmask carried alongside the scaled matrix. Presolve, branching and cut separation query it in $O(1)$ instead of re-deriving it downstream.

Note: scaling (D1, D2) touches every column, integer or not — it doesn't affect which points are integer-feasible, only the relaxation's numerics — but D2's column scaling must be inverted (Step 12) before an integrality check on x means anything, and any bound projection performed **before** that inversion must use the scaled bounds, not the original ones (see Step 12).

Step 2 — MILP-Aware Presolve (CPU)

All of LP Step 2 (singleton/doubleton elimination, FBT/DBT, duplicate row/column merging) runs unchanged and to a fixed point first — generic reductions are cheaper and often expose integer structure. Then loop the following, interleaved with the LP reductions, until nothing fires:

1. **GCD / Coefficient Tightening.** Rows keep their native inequality/equality sense throughout this pipeline — no standard-form conversion into equalities-with-slack-columns happens anywhere in presolve or beyond; "Ax = b" elsewhere in this document is shorthand covering rows of mixed sense, not evidence of slack augmentation. Under that representation: for a row where every structural variable is integer-constrained and every coefficient A_j is itself an integer, let $g = \gcd(A_1, \dots, A_n)$. Any achievable integer left-hand side is a multiple of g , so the RHS tightens without cutting off a single integer point — by direction:

$$\sum A_j x_j \leq b: \quad b \leftarrow g \cdot \text{floor}(b / g)$$

$$\sum A_j x_j \geq b: \quad b \leftarrow g \cdot \text{ceil}(b / g)$$

$$\sum A_j x_j = b: \quad \text{infeasible outright if } b \text{ is not itself a multiple of } g$$

Both preconditions — integer variables and integer coefficients — are required in every case: a row with integer variables but fractional coefficients admits no such gcd argument.

2. **Probing.**
 - For each binary $x_j \in B$: temporarily fix $x_j = 0$ and propagate via FBT/DBT (Step 1's reduction machinery, reused); repeat for $x_j = 1$.
 - Propagation from one branch alone proves infeasibility $\rightarrow$ fix x_j to the other value globally.
 - A bound on some x_k tightens identically under both branches $\rightarrow$ apply it unconditionally, regardless of x_j .
 - $x_j = 1$ forces $x_k = 0$ (or any such pairwise implication) $\rightarrow$ record a directed edge in an implication graph, consumed by clique detection below and later by branching.

Probing is the single most expensive presolve pass ($2 \cdot |B|$ propagation sweeps) — cap it by degree or a time budget on large models rather than running it exhaustively.

3. **Clique / Conflict-Graph Detection.** Build a conflict graph over binaries (and their complements) from set-packing rows ($\sum x_j \leq 1$) and from probing's implication graph. Extend cliques greedily — a maximal clique is a valid $\sum x_j \leq 1$ inequality even where it wasn't a literal input row. Store the clique table for direct reuse in Step 7, not just as a presolve reduction.
4. **Symmetry Detection (optional, criteria stated explicitly).** Flag a candidate orbit only under a concrete, checkable rule: two columns j, k are symmetric if they have identical sparsity pattern in A (same row support), identical coefficients on that support, identical objective coefficient ($c_j = c_k$), and identical bounds ($l_j, u_j = l_k, u_k$) — verified by canonical hashing of each column's (support, coefficient, c_j , l_j , u_j) tuple, the same xxHash64/MurmurHash3 machinery Step 2's duplicate-merging pass already uses, extended to include the objective and bounds in the hash key. Where a matching orbit is found (common in scheduling and assignment

structures — identical parallel units, identical time slots), add lexicographic-ordering constraints on one representative per orbit to cut symmetric duplicate subtrees before Branch-and-Bound ever explores them. This hash-based rule is deliberately narrower than a general graph-automorphism search — it will miss symmetry that isn't literal column-for-column duplication (e.g. symmetry only visible after a linear reformulation), which a full automorphism search would catch but at a cost too high to run unconditionally on industrial-scale models.

The same hashing approach extends to **row (constraint) symmetry**: two rows i, k are candidates for interchangeability if they have identical sparsity pattern (same column support), identical coefficients on that support, identical RHS $b_i = b_k$, and identical constraint sense ($\leq / \geq / =$) — hashed the same way as columns above. Interchangeable rows are common in exactly the target domain (identical time-slot or identical parallel-unit constraints in scheduling models), and were part of this step's original scope; omitting row symmetry while only formalizing column symmetry would silently narrow what this step catches. Where a row orbit is found, the same lexicographic-ordering treatment applies to break it before Branch-and-Bound explores symmetric duplicate subtrees.

5. **Presolve Transaction Log (M_{presolve} — extended).** As in LP, plus: every probing-forced fixing, every GCD/coefficient tightening, and any clique/symmetry rows added (which get dropped, not un-presolved, on the way back — they have no counterpart in the original formulation).

Step 3 — GPU Warm-Start: Restarted PDLP on the Root Relaxation (GPU) — root only

Mechanically identical to LP Step 3: transfer $A_{\text{red}}, b_{\text{red}}, c_{\text{red}}$ (integrality dropped — this is the LP relaxation of the presolved root) into VRAM; run the Primal-Dual Hybrid Gradient (PDHG) loop — extrapolated primal update, dual/primal projection steps, Malitsky–Pock step sizes, adaptive restarts on stalled or oscillating primal-dual gap — to $\epsilon \approx 10^{-3} - 10^{-4}$; transfer the face point back to host. Implementation is via **hand-written, native CUDA SpMV/AXPY kernels as the primary path**, matching the LP pipeline's own current direction.

Fallback contingency — stated up front, not as an afterthought

Native kernels are the primary implementation target; this carries real, non-trivial engineering risk on unstructured sparse matrices (memory coalescing, warp-level load balance) relative to a mature vendor library. If profiling against target-scale industrial and MIPLIB-class instances shows native kernels underperform cuSPARSE/cuBLAS by a material margin, the SpMV/AXPY layer inside this step falls back to those vendor primitives without changing anything upstream (Steps 1–2) or downstream (Steps 4 onward) of this step — the fallback is a substitution at the arithmetic layer only, not an architectural change.

Scope note. This runs exactly once per MILP solve. Every subsequent node — there may be millions — reoptimizes via warm-started Dual Simplex (Step 5) instead of returning here; see the callout under Step 5.

Step 4 — Crossover: Basis Recovery (CPU) — root only

Identical to LP Step 4 (status classification of $(\bar{x}_j, \bar{y}_i, \bar{s}_j)$; Megiddo/Andersen primal/dual push) applied to the root's PDL face point, producing the root basis B_0 . This is the only crossover the whole MILP solve performs — every child node's basis descends from an ancestor's by Dual Simplex re-optimization, never by a fresh crossover.

Step 5 — High-Precision Dual Revised Simplex — Shared Root & Node Engine (CPU)

The exact machinery of LP Step 5 — Markowitz-reordered LU ($B = P_L \cdot L \cdot U \cdot P_R$), Devex/Steepest-Edge pricing, Harris two-pass ratio test, Forrest-Tomlin factor updates, full refactorization every 500–1,000 pivots — called from two places:

1. **Root call, once:** from B_0 (Step 4), drive to 10^{-8} – 10^{-12} , exactly as in the LP pipeline.
2. **Node calls, per Branch-and-Bound node:** from the parent's converged basis, with exactly one variable's bound tightened (the branching decision just made). Since only one bound moved, the parent basis is dual-feasible or close to it — Dual Simplex typically needs a handful to a few dozen pivots to reconverge, versus a cold start's $O(m)$ or worse.

Why this, and not a fresh Step 3/4 GPU restart, at every node

A first-order GPU method's PCIe round trip alone costs more wall-clock time than a warm-started re-optimization from a near-optimal basis, and PDHG's 10^{-4} accuracy floor is a worse starting point than a machine-precision basis sitting one bound-flip away. The GPU pipeline earns its keep exactly once, where the relaxation is solved cold with nothing to warm-start from; everywhere else in the tree, warm-started Simplex wins on speed, accuracy and determinism simultaneously.

Step 6 — In-Loop Degeneracy & Ill-Conditioning Control (inside every Step 5 call)

Unchanged from LP Step 6: bound perturbation as the primary anti-cycling mechanism, Bland's rule reserved as a rare deterministic fallback, iterative refinement triggered when $\kappa(B)$ spikes and $\|b - B \cdot x_B\|$ exceeds tolerance.

MILP-specific amplifier — cost left explicitly unquantified, flagged rather than assumed away

Every branching decision manufactures a fresh, tightly bound vertex, and a newly tightened bound very often binds exactly at the new LP optimum — so this control fires markedly more often per node in MILP than in one standalone LP solve. The mechanism itself is unmodified; only its trigger frequency changes, and that frequency is not bounded here. In the worst case, per-node perturbation/refinement overhead compounds across a tree of millions of nodes into a real, currently unmeasured cost. This should be profiled empirically against representative refinery-

scheduling and MIPLIB instances before any wall-clock claim is made about the tree-search phase — it is not safe to assume this cost is negligible by analogy with the single-solve LP case.

Step 7 — Root Cutting-Plane Loop (Branch-and-Cut: Root Processing) (CPU, GPU-assisted separation)

Entered once, immediately after the root's Step 5 converges.

1. **Integrity check.** Scan x^* for $j \in I$ with $|x_j^* - \text{round}(x_j^*)|$ above tolerance. None found $\rightarrow$ the root relaxation is already MILP-optimal, skip to Step 12 (rare, but cheap to check).
2. **Cut separation**, drawing on several families:

- *Gomory Mixed-Integer (GMI) cuts* — the workhorse, generic to any fractional integer row. From the optimal tableau row for basic fractional $x_i, i \in I$:

$$x_i + \sum_{(j \in N)} \bar{a}_j x_j = \bar{b}, \quad f_0 = \bar{b} - \text{floor}(\bar{b}) \quad (0 < f_0 < 1)$$

integer nonbasic j , $f_j = \bar{a}_j - \text{floor}(\bar{a}_j)$:

$$\alpha_j = f_j \quad \text{if } f_j \leq f_0$$

$$\alpha_j = f_0 \cdot (1 - f_j) / (1 - f_0) \quad \text{otherwise}$$

continuous nonbasic j :

$$\alpha_j = \bar{a}_j \quad \text{if } \bar{a}_j \geq 0$$

$$\alpha_j = -\bar{a}_j \cdot f_0 / (1 - f_0) \quad \text{otherwise}$$

$$\text{cut: } \sum_j \alpha_j x_j \geq f_0$$

- *Mixed-Integer Rounding (MIR) cuts* — derived by rounding a single (possibly aggregated) row against its integer/continuous split; a superclass several structured families specialize.
- *Lifted cover cuts* on knapsack rows ($\sum a_j x_j \leq b, x$ binary): find a minimal cover C with $\sum_{(j \in C)} a_j > b$, start from $\sum_{(j \in C)} x_j \leq |C| - 1$, then lift the non-cover variables back in for a tighter, often facet-defining, inequality.
- *Clique cuts*, read directly from Step 2's conflict-graph table — no re-detection needed at the root.
- *Flow-cover cuts* — unlike the other families above, these are not pre-flagged during Step 2; detect candidate network-flow-like row blocks directly during separation by scanning for rows with the conservation-law signature (coefficients restricted to $\{+1, -1\}$, each column touching at most two such rows), then apply flow-cover lifting to any resulting knapsack-structured sub-row. This is stated explicitly here because Step 2 performs no structural flow-detection pass of its own — earlier drafts of this document referenced one that was never specified, which this revision removes.

GPU-assisted separation: scoring a large candidate cut/row set for violation against x^* is an SpMV-shaped batch operation (thousands of $\text{candidate_cut} \cdot x^*$ comparisons at once) — a genuine GPU fit, unlike the combinatorial work in Steps 8–9. Implemented via the same SpMV kernel machinery as Step 3

(native-first, vendor-fallback), applied to a distinct matrix: the stacked candidate-cut rows for that separation round, rebuilt each round as the candidate set changes, rather than the fixed Q_s/A_s from Step 1. It is the same kernel type doing the same operation — a sparse matrix applied to a dense vector — just against different, transient data; not a different class of kernel.

3. **Cut management.** Score by violation magnitude, orthogonality to already-accepted cuts (reject near-parallel duplicates), and the new row's sparsity/conditioning; keep the top-k per round in a global cut pool shared with the tree (Step 9).
4. **Re-optimize.** Append accepted cuts as new rows with slack and re-enter Step 5 as a warm start from the pre-cut basis — no re-crossover, no GPU restart.
5. **Iterate 1–4** for a capped number of rounds (default: 20 rounds at the root); stop early once per-round violation improvement or fractional-gap closure drops below a 1% relative threshold (the classical cutting-plane "tailing-off" — chasing it indefinitely wastes time better spent branching).

A lighter version of point 2 above (typically GMI and cover cuts only, no full separator sweep) may optionally re-fire at selected tree nodes — see Step 9, point 5 — but the bulk of the separation work and benefit happens here, once, at the root.

Step 8 — Branching: Variable Selection & Node Creation (CPU)

Entered once root cutting (Step 7) stalls with x^* still fractional.

1. **Candidates:** $j \in I$ with x_j^* fractional.
2. **Branching rule — reliability-branching hybrid:**
 - *Strong branching* (used until a variable's history is trustworthy): for a short candidate shortlist, tentatively solve both child LPs to a limited pivot count (Step 5, CPU, discarded after scoring) and rank by predicted bound improvement — the most informative choice per node, and the most expensive.
 - *Pseudocost branching* (used once reliable): maintain a running per-variable, per-direction average objective degradation from actual historical branches; score candidates from these averages with no trial solves — far cheaper, carries the bulk of the tree.
3. **Node creation.** Split the chosen x_j at x_j^* into two children, $x_j \leq \text{floor}(x_j^*)$ and $x_j \geq \text{ceil}(x_j^*)$; push both onto the open-node pool (Step 9), each carrying a reference to the parent's basis for warm-starting.

Step 9 — Node Selection & Tree Traversal (multi-core CPU — not GPU)

1. **Open-node pool**, ordered by a selection strategy:
 - *Best-bound:* always expand the globally most favorable open LP bound — minimizes the node count needed to *prove* optimality, slow to find an early incumbent.
 - *Plunging / DFS:* dive one branch to a leaf — finds incumbents fast, weak at bound improvement.

- *Best-estimate (default)*: rank by LP bound combined with a pseudocost-projected integer-objective estimate; shift toward pure best-bound once a good incumbent exists and the search's job changes from "find a solution" to "prove this one is optimal."
2. **Parallelization.** Independent subtrees dispatch across a work-stealing CPU thread pool — deliberately CPU-only. Tree traversal is irregular, pointer-chasing, branch-heavy control flow with unpredictable per-node cost: the opposite of the uniform, data-parallel SpMV/AXPY work GPU threads are built for. GPU stays confined to Steps 3 and 7's numerical kernels; the tree itself never touches it, and no part of variable selection or node scheduling runs on GPU.
 3. **Bounding / pruning.** Fathom a node without further branching if its relaxation is infeasible, its bound is no better than the current incumbent (within tolerance), or its solution is already integer-feasible — in the last case, atomically compare-and-update the incumbent objective together with its snapshot as a single guarded operation: if this node's objective improves on the best known, replace both the recorded incumbent value and the stored snapshot — this node's branched bound set (l_node , u_node) plus its lightweight basic-index set and dual vector, not the full LU factorization, which can be reconstructed on demand from the index set if the diagnostic in Step 12 is ever invoked — under the same lock/atomic mechanism already guarding the shared incumbent value in this work-stealing parallel search. Updating the objective and the snapshot as two separate, unsynchronized writes would let two threads' near-simultaneous improving solutions interleave into a snapshot that doesn't match the recorded incumbent; the update must be a single guarded operation, not two. Exactly one snapshot is retained at a time, overwritten whenever a better incumbent supersedes the previous one.
 4. **Reduced-cost fixing.** At every node, use the relaxation's reduced costs together with the current incumbent-bound gap to permanently fix variables whose best-case contribution cannot beat the incumbent; propagate the fixing to every descendant of that node.
 5. **Optional local re-separation.** A restricted subset of Step 7's separators (typically GMI and cover cuts only) may re-fire at selected nodes — shallow depths, or subtrees stalled for many consecutive nodes without bound improvement — capped in both frequency and cut count so separation overhead never swamps node throughput.

Step 10 — Primal Heuristics (CPU, root + capped frequency in-tree)

1. **Diving family** (fractional / coefficient / pseudocost diving): repeatedly fix the most promising fractional variable by a cheap rule and warm-start-resolve (Step 5) until integer-feasible or infeasible — a single greedy descent, not a search.
2. **Feasibility pump.** Alternate rounding the current LP point to the nearest integer point with re-solving an L1-distance-minimizing LP back toward that rounded target, iterating until the rounded and relaxed points coincide — the standard fallback when structure resists simple diving; perturb on detected cycles.
3. **RINS / RENS.** RINS fixes variables that agree between the incumbent and the current node relaxation; RENS fixes variables already near-integral in the relaxation alone, with no incumbent required. Either way, the freed residual sub-MILP is re-solved with the same Branch-and-Cut machinery (Steps 7–9) at a restricted scale.

4. **Scheduling.** Run the full suite once at the root, right after the first cutting round; thereafter trigger diving/RINS at a capped node frequency (default: every 100 nodes, or on a 5-minute wall-clock interval, whichever comes first) — heuristics cost real LP re-solves and must not dominate tree-search time.

Step 11 — Incumbent, Gap & Termination (CPU)

1. **Track** the incumbent x_{inc} (best integer-feasible objective found) and the global best bound — the best LP bound across every still-open node, the tree's current proof of how much better a solution could theoretically get.
2. **Gap** = $|\text{best_bound} - \text{incumbent_obj}| / (|\text{incumbent_obj}| + \epsilon)$.
3. **Terminate** on whichever fires first:
 - open-node pool empty → incumbent is proven optimal, gap = 0;
 - $\text{gap} \leq \text{target tolerance}$ (default: 10^{-4} relative) → accept, report the certified gap;
 - default operational budget reached — 10^6 nodes, or a wall-clock cap sized to the deployment's actual use case (illustratively: seconds-to-low-minutes for a hot-started re-solve of a lightly-perturbed instance per the Hot-Starting Engine, versus hours for a cold-started root solve on a million-variable refinery-scale instance) — whichever comes first, both configurable via the API → report the best incumbent and best proven bound honestly as bounded, not claimed optimal.

Note: the 10^6 -node figure and the illustrative wall-clock ranges above are starting points, not tuned values — given as worked examples across the two operational modes this pipeline actually supports (cold-start vs. hot-start), rather than as one blanket number that would be meaningless across both. Neither has been set from empirical runs against representative instances yet.

Step 12 — Un-Presolve, Bound Projection, Un-Scale & Final Verification (CPU)

Same strict Last-In, First-Out order as LP Step 7 — undo presolve, then project onto bounds, then invert scaling, because scaling was applied first (Step 1) and presolve operated on the already-scaled system (Step 2) — replayed on the final incumbent x_{inc} , plus MILP-specific corrections below.

1. **Un-presolving.** Replay the extended M_{presolve} (including Step 2's probing fixings and coefficient tightenings) in reverse chronological order. The result, $x_{\text{red_orig}}$, is still in the **scaled** coordinate system — un-presolving reverses Step 2's reductions, not Step 1's scaling.
2. **Bound projection — corrected to operate in the scaled space (Revision 2 fix).** Substitutions replayed during un-presolving can reintroduce small numerical bound violations ($\sim 10^{-7}$) that were not present in the reduced space. These must be projected out **before** un-scaling, using the **scaled** bounds $l_s = D2^{-1}l$, $u_s = D2^{-1}u$ carried from Step 1 — **not** the original l , u . Projecting a still-scaled vector directly onto the original, unscaled box (as an earlier draft of this document

did) is dimensionally invalid: it mixes two different coordinate systems in a single operation and silently corrupts the result. The corrected step is:

```
| x_red_orig ← proj_[l_s, u_s]( x_red_orig )      (scaled bounds –
| x_red_orig has not been un-scaled yet)
```

3. **Un-scaling.** Only now invert Step 1's scaling on the primal vector:

```
| x_final = D2 · x_red_orig
```

The corrected certificate below (item 4) needs only x_{final} — no globally un-scaled dual vector is part of that certificate, since MILP has no strong duality to certify with one. If the API additionally reports a dual/reduced-cost vector for diagnostic or downstream-tooling purposes, it is sourced from the incumbent's leaf-node snapshot (item 4, third bullet below) and **may be un-scaled by D1 for display**, exactly as a rigorous dual would be — un-scaling is a units conversion, not a validity claim. What must never happen is presenting that un-scaled value as if it were a solved-problem certificate: it is always labeled **non-rigorous, final-node-only, relative to that node's branched bounds**, whether shown in scaled or un-scaled units.

4. **Final verification — rescoped to what a MILP incumbent can actually certify (Revision 2 fix).** A MILP has no strong duality: there is no dual certificate analogous to LP's KKT system that proves overall MILP optimality, and the incumbent's associated (y, s) pair belongs to its own leaf node's *branched* bounds, not the original problem's bounds — checking it against the original, unbranched A, b, c (as an earlier draft of this document did for the dual residual and complementary-slackness terms) does not certify what it claims to. The actual optimality statement for a MILP is Step 11's gap between best bound and incumbent, already tracked continuously — this step verifies **feasibility**, not a spurious duality gap:
 - *Primal residual* $\|Ax_{\text{final}} - b\|_{\text{inf}} / (1 + \|b\|_{\text{inf}}) \leq 10^{-8}$ — checked against the original, unpresolved, unscaled A, b : this is the meaningful feasibility statement, since it doesn't depend on any dual object.
 - *Bound feasibility* $\max_j \max(0, l_j - x_{j,\text{final}}, x_{j,\text{final}} - u_j) \leq 10^{-8}$ — an explicit check against the original, unscaled box, independent of trusting item 2's scaled-space projection to have carried through un-scaling correctly. Cheap ($O(n)$) and included specifically because a scaled/unscaled mismatch at exactly this boundary was the defect this revision exists to close — verify, don't assume an earlier step silently got it right.
 - *Integrity* $\max_{(j \text{ in } I)} |x_{j,\text{final}} - \text{round}(x_{j,\text{final}})| \leq 10^{-6}$ — the check that actually distinguishes a certified MILP solution from a merely LP-feasible one.
 - *Optional leaf-node sanity check* (diagnostic only, not an optimality certificate, and only meaningful if a snapshot was retained per Step 9): using the single retained $(l_{\text{node}}, u_{\text{node}}, \text{basic-index-set}, \text{duals})$ snapshot for the final incumbent, check that leaf node's own LP-KKT residuals — dual residual and complementary slackness — against **its own branched bounds**, entirely within that node's own scaled space. This value is never combined with, or un-scaled alongside, the global x_{final} above. It confirms the leaf LP was solved correctly at that node; it says nothing about MILP optimality, which remains Step 11's job alone.
5. **Report** alongside the solution whether it is proven-optimal or budget-limited, and the realized gap either way (Step 11).

Consolidated Pipeline Summary

#	Phase	Method(s)	Where	When it fires
1	Ingestion & Scaling	CSC/CSR parsing, integrality tagging, Ruiz Equilibration, Pock-Chambolle preconditioning	CPU	Once, first
2	MILP Presolve	LP reductions + GCD/coefficient tightening (integer coeffs required), probing, clique detection, hash-criteria symmetry detection, extended M_presolve	CPU	Once, loop to fixed point
3	Root Warm-Start	Restarted PDLP (PDHG, from-scratch); native CUDA kernels primary, cuSPARSE/cuBLAS fallback if profiling requires it	GPU	Once, root only, to $\epsilon \approx 10^{-4}$
4	Root Crossover	Megiddo/Andersen basis purification	CPU	Once, root only
5	Exact Solve	Dual Revised Simplex, Markowitz LU, Devex/Steepest-Edge, Forrest-Tomlin updates	CPU	Root: once to 10^{-8} – 10^{-12} ; every node: warm-started re-optimization
6	Degeneracy / Stability	Bound perturbation (primary), Bland's rule (fallback), Harris ratio test, iterative refinement — frequency amplified per node, cost unquantified	CPU	Reactively, inside every Step 5 call
7	Root Cutting Planes	GMI, MIR, lifted cover, clique, flow-cover (detected at separation time) — capped 20 rounds, 1% tailing-off threshold	CPU (+GPU-assisted scoring)	Root, capped number of rounds
8	Branching	Strong / pseudocost reliability-branching hybrid; two-child node creation	CPU	Every node with a fractional solution
9	Tree Search	Best-bound / plunging / best-estimate node selection; work-stealing parallel traversal; reduced-cost fixing; pruning; optional local re-separation	Multi-core CPU	Continuously, until pool empty or terminated
10	Primal Heuristics	Diving family, feasibility pump, RINS/RENS — capped every 100 nodes / 5 min	CPU	Root, then capped frequency in-tree
11	Incumbent / Gap	Bound-vs-incumbent gap tracking; default 10^{-4} gap, 10^6 -node, wall-clock budget	CPU	Continuously
12	Post-Solve	Un-presolve, scaled-space bound projection, un-scaling, feasibility + integrality verification (no invalid dual certificate)	CPU	Once, last, on the final incumbent

On Sovereignty: From-Scratch Cut, Branching and Heuristic Logic

Every cut family in Step 7 (GMI, MIR, lifted cover, clique, flow-cover), the branching rules in Step 8 (strong, pseudocost, reliability), and the heuristics in Step 10 (diving, feasibility pump, RINS/RENS) are classical, published mathematics — implemented here from the original formulas (Gomory; Balas; Marchand & Wolsey; Van Roy & Wolsey; Achterberg, Koch & Martin; Fischetti, Glover & Lodi; Danna, Rothberg & Le Pape), not adapted from any existing open-source MIP solver's source tree, consistent with the problem statement's "built from scratch from mathematical foundation" mandate — which scopes the solver algorithm itself, not the GPU arithmetic primitives underneath it. The numerical layer (Steps 3 and 7's separation scoring) targets native CUDA kernels as the primary implementation, with cuSPARSE/cuBLAS as a stated fallback if profiling shows the native kernels underperforming; either way, no third-party LP/MIP solver library sits underneath any decision-making stage of the pipeline.

Honest cost — what remains unverified

As with the LP pipeline's own disclosure: correctly implementing the formulas above is necessary but not sufficient. A mature commercial solver's edge on real MIPLIB-class instances comes as much from years of cut-selection tuning, pseudocost calibration, and heuristic scheduling as from the base algorithms themselves. The operational defaults stated in Steps 7, 10 and 11 (round caps, node/time budgets, re-separation frequency) are starting points, not validated values — none of this has been run against MIPLIB, Netlib or representative refinery-scheduling instances yet. Expect a first correct implementation to be numerically sound and feasibility-certified well before it is competitive on wall-clock time; that gap is closed by empirical benchmarking, not by further derivation.

Key Structural Note

Branch-and-Bound is not the LP pipeline re-run from scratch at every node. It is a thin combinatorial shell that calls the LP core's expensive, once-only stages — GPU warm-start (Step 3) and Crossover (Step 4) — exactly once, at the root, and its cheap, re-enterable stage — Dual Revised Simplex with in-loop degeneracy control (Steps 5–6) — at every one of the potentially millions of nodes that follow. Cutting-plane separation is heaviest at the root, where it tightens the bound for the whole tree at once, and is reused sparingly thereafter through the shared cut pool rather than fully re-derived at each node. Within Branch-and-Cut, GPU acceleration stays confined to genuinely data-parallel numerical work — the root relaxation (Step 3) and batched cut-violation scoring (Step 7) — while variable selection, strong-branching trial solves, and tree traversal itself are CPU territory throughout: strong branching's trial solves route through Step 5 on CPU, not through any GPU kernel, and no part of node scheduling touches the GPU.